Combinatorial Algorithms (Math 482), 2026 Spring Problem Set 4

Due by Wednesday, February 18, 2026.

1. **Greedy approach for rod-cutting.** Recall that in the Rod Cutting problem, we are given a rod of size n and a price vector $P[1..n]$, and we want to maximize $\sum_{i=1}^{k} n_i P[i]$ subject to the constraint $n = \sum_{i=1}^{k} n_i$.

   Oliver suggests the following algorithm: Compute the *unit price* $U[i] = P[i]/i$ for all $i = 1, .., n$. Find a maximum $U^* = \max_{1 \le i \le n} U[i]$. Suppose the maximum unit price is $U^* = U[i^*]$. Cut the rod of length n into the maximum number of pieces of length i^*.

   This algorithm produces $\lfloor n/i^* \rfloor$ pieces of length i^* and up to one "leftover" piece of length $n - \lfloor n/i^* \rfloor \cdot i^*$.

   Construct an instance of the rod-cutting problem, where this algorithm does not find the optimal solution. Compare the result of Oliver's algorithm with the optimum.

2. **Minimum subset sums.** We are given an array of positive integers $A[1..n]$ and a threshold W. Find a minimum set of integers from $A[1..n]$ such that their sum is at least W, or report that no such subset exists. Formally, we want to find a binary array $B[1..n]$ that minimizes $\sum_{i=1}^{n} B[i]$ subject to the constraint that $\sum_{i=1}^{n} A[i] \cdot B[i] \ge W$.

   Olivia suggests the following algorithm: Assume that the array $A[1..n]$ is sorted in nondecreasing order (we can sort it in $O(n \log n)$ time). For $i = 1, \ldots, n$, compute the prefix sums $S[i] = \sum_{j=1}^{n} A[j]$. If $S[n] < W$, then report that there is no solution. Otherwise, find the smallest index i^* such that $S[i] \ge W$; set $B[j] = 1$ for all $j \le i^*$ and $B[j] = 0$ for all $j > i^*$; and return the array $B[1..n]$.

   Prove or disprove that Olivia's algorithm always finds the optimum solution. What is the running time of this algorithm?

3. **Subset Sum with Conflict Pairs.** Consider the following modified version of the subset sum problem.

   We are given an array $A[1..n]$ of positive integers, a positive integer W, and a conflict list $L = [(x_1, y_1), (x_2, y_2), \ldots, (x_k, y_k)]$. We need to choose a subset of the numbers in the array to maximize the sum subject to the constraints that the sum is at most W, and we cannot choose both $A[x]$ and $A[y]$ if (x, y) is on the conflict list.
   Formally, find a binary array $B[1..n]$ that maximizes $\sum_{i=1}^{n} A[i] \cdot B[i]$ subject to the constraints that $\sum_{i=1}^{n} A[i] \cdot B[i] \le W$ and $B[x] \cdot B[y] = 0$ if (x, y) is on the list L.

   Design a *dynamic programming* algorithm for this problem. Find a recurrence relation, write pseudo-code for the algorithm, and analyze its runtime.

4. **Longest palindrome subsequence.** A *palindrome* is a nonempty string over some alphabet that reads the same forward and backward. Examples of palindromes are all strings of length 1, `civic`, `racecar`, and `aibohphobia` (fear of palindromes).

   Design a *dynamic programming* algorithm to find the longest palindrome that is a subsequence of a given input string. For example, given the input `character`, your algorithm should return `carac`. What is the running time of your algorithm?